DevOps Question Bank
Behavioral & Project Experience

19 real interview questions - de-duplicated, topic-wise, Basic / Advanced / Scenario-Based
Every answer: plain-English idea · technical answer · example · what to say in the interview

By Priyanka Gupta — 25 September 2026

How to use this question bank

Each question has four parts:

In simple words (green box) - a plain-English analogy. Read this first; if you understand it, the technical part becomes easy to remember.

Detailed answer + Example - the real technical answer, with commands / YAML / code you can show.

What to say to the interviewer (blue box) - 2-3 lines to say out loud. Practise these aloud.

Questions that were asked more than once across the lists were merged into one (the other wordings are shown in grey as 'Also asked as'). Language-specific (Python/PHP) and non-DevOps questions were removed. Where a pasted 'model answer' was wrong, the corrected version is given.

Suggested order: all Basic questions first, then Scenario-Based (most interviews are troubleshooting-heavy), then Advanced. Use the PDF bookmarks panel to jump to any question.

Topic	Basic	Advanced	Scenario	Total
Behavioral & Project Experience	9	0	10	19

Questions in this topic

Basic **4**

Q1. Tell me about yourself. . . . 4

Q2. What are your daily responsibilities as a DevOps engineer? . . . 4

Q3. Walk me through your current project architecture and your role in it. . . . 5

Q4. Which DevOps tools have you worked with in the last 2 years? . . . 6

Q5. Which AWS services are you proficient in, and how many have you worked with? . . . 7

Q6. What have you achieved in your career so far? . . . 7

Q7. How much experience do you have handling databases? . . . 8

Q8. How do you use Linux in your DevOps work, and which distributions have you used? . . . 9

Q9. Can you describe the CI/CD workflow in your project? . . . 9

Scenario-Based **10**

Q10. Tell me about a recent outage or production incident you handled. What did you learn from it? . . . 10

Q11. Tell me about a mistake you made in a DevOps project and what you learned from it. . . . 11

Q12. What is the hardest technical issue you have faced, and how did you solve it? . . . 11

Q13. Tell me about a time you had to deliver something quickly without all the time or resources you needed. . . . 12

Q14. What would you do if you had to work on a tool or technology you have never used? Share a time you picked up a new tool quickly. . . . 12

Q15. How would you convince your team or manager to adopt a new tool or process? . . . 13

Q16. How do you handle conflict or disagreement within your team? . . . 14

Q17. How do you work across teams (developers, QA, platform, business)? . . . 14

Q18. Give an example of taking ownership beyond your assigned task. . . . 15

Q19. Can you design, secure, scale, troubleshoot and own production systems? . . . 15

Behavioral & Project Experience

Basic

Q1. Tell me about yourself.

Also asked as: Introduce yourself.

> **In simple words:** The interviewer wants a 60-90 second story: who you are, what you work on now, one or two results you are proud of, and why you want this role. It is not your life history.

Detailed answer

Use the formula **Present -> Proof -> Future**:

- **Present** - your role and the platform you support.
- **Proof** - two concrete achievements with results.
- **Future** - what you want next and why this role fits.

Example: Sample answer - adapt to your own words:

```
I have around 5 years in IT, and currently I work on the operations and monitoring
side of a smart-metering platform at a smart-energy technology company. The platform runs on Amazon
    EKS
across four production environments, with services like HES, MDMS and WFM, backed
by RDS, ElastiCache and Kafka (MSK).

My focus is observability and reliability: Zabbix, Prometheus/Alertmanager, Grafana
and Loki, SLA dashboards and reports, and alert routing to Microsoft Teams. For
example, I built a zero-cost Teams alerting path with Lambda relays when our planned
on-call tool became unavailable, and I found and fixed a bug where new alert media types
had empty message templates, which had silently blocked alert delivery.

Now I want to grow into a full DevOps role - CI/CD, Terraform and Kubernetes
operations - and this role is a good fit because it combines that with production
ownership.
```

> ***What to say to the interviewer:*** *Keep it to about 90 seconds: current role and platform, two proof points with results, and why you want this specific role. End on something that invites a follow-up question you are ready for.*

Q2. What are your daily responsibilities as a DevOps engineer?

Also asked as: Hiring-manager round - describe your day-to-day responsibilities. What are your daily tasks?

> **In simple words:** They want to see that your day is about keeping production healthy and improving it, not just doing tickets. Show a rhythm: check health, fix issues, improve something, communicate.

Detailed answer

Structure it as a routine:

- **Morning health check** - review overnight alerts, SLA dashboards, daily health reports for all environments.

- **Incident and alert handling** - triage alerts, find root cause, fix or escalate, and verify that alerts actually reach the Teams channels.
- **Monitoring improvements** - new dashboards, alert rules, thresholds, reducing noise, segregating alerts by category.
- **Automation** - scheduled Lambda reports, health probes, scripts that remove manual checks.
- **Reporting** - daily and monthly SLA reports with downtime and root causes.
- **Tickets and collaboration** - Jira stories and subtasks, working with developers and the platform team.
- **Cost and hygiene** - finding orphaned volumes, unused buckets, over-sized resources.

Example: Sample answer - adapt to your own words:

```
My day starts with checking overnight alerts and the daily health reports for our
four environments. Then I handle any open incidents - for example a disk or memory
alert on a monitoring server. The rest of the day is improvement work from Jira:
building or fixing alert rules, Grafana dashboards, automation for reports, and
preparing SLA reports that explain every minute of downtime with a root cause.
```

> ***What to say to the interviewer:*** *Describe a routine - health checks, incident handling, improvement work, automation and reporting - with one real example for each. Interviewers remember the example, not the list.*

Q3. Walk me through your current project architecture and your role in it.

> **In simple words:** They want to see if you understand the whole system, not only your piece. Draw it from the user's request inward, then say which boxes you own.

Detailed answer

Explain in layers, high level, no internal IDs:

- **Business** - smart-metering platform for power utilities: meters send readings, the platform collects, stores, bills and manages field work.
- **Applications** - HES (Head End System: talks to meters, collects data), MDMS (Meter Data Management: validates, stores, processes readings), WFM (Workforce Management: field jobs), running as microservices on **Amazon EKS**.
- **Data layer** - Amazon RDS (PostgreSQL) for databases, ElastiCache (Redis) for caching/sessions, Amazon MSK (Kafka) for streaming meter events. One environment runs self-managed PostgreSQL/Redis/Kafka on EC2.
- **Environments** - four production environments for different utility projects, each in its own AWS account or setup.
- **Monitoring layers** - Zabbix for host, service and SLA monitoring; Prometheus + Alertmanager inside the clusters; Loki for logs; Grafana dashboards; CloudWatch alarms for AWS services; alerts delivered to Microsoft Teams channels.

Example:

```
Meters -> HES (EKS) -> Kafka (MSK) -> MDMS (EKS) -> RDS PostgreSQL
                           |                |
                      Redis cache      WFM / billing / APIs -> ALB -> users
Monitoring: Zabbix + Prometheus/Alertmanager + Loki + CloudWatch -> Grafana -> Teams
```

My role: I own the observability and alerting layer across all four environments - dashboards, alert rules, SLA reporting, report automation, health checks - and I am the first responder for monitoring and infrastructure alerts.

> ***What to say to the interviewer:*** *Walk from the meter to the user through HES, Kafka, MDMS and the databases, then describe the monitoring layer, and finish with a clear statement of what you own. Be ready to go one level deeper on any box you mention.*

Q4. Which DevOps tools have you worked with in the last 2 years?

Also asked as: Which IaC tools have you used - Terraform, Ansible, Puppet? Tell me about your cloud project work.

> **In simple words:** They are checking depth, not the length of the list. Group tools by category and say honestly where you are hands-on versus learning.

Detailed answer

Group the answer:

Category	Hands-on in work	Practising / learning
Monitoring & logs	Zabbix, Prometheus, Alertmanager, Grafana (incl. API-built dashboards), Loki	OpenTelemetry, Tempo
Cloud (AWS)	EKS, EC2, Lambda, EventBridge, CloudWatch, SNS, RDS, ElastiCache, MSK, S3, IAM	Route 53, VPC design
Containers & K8s	kubectl for troubleshooting, reading pod logs/events	Helm, writing manifests
IaC & CI/CD	-	Terraform, Jenkins, GitHub Actions, Argo CD
Config management	-	Ansible (Puppet and Chef: concepts only)
Scripting & OS	Linux, Bash, cron/scheduled jobs	Ansible
Collaboration	Jira, Confluence, Git	-

For the IaC question, be clear on the difference: Terraform provisions infrastructure (declarative, keeps state), while Ansible, Puppet and Chef configure servers (Ansible is agentless and push-based; Puppet uses agents and pulls). Then say honestly which ones you used in production and which in labs. For cloud project work, point to the AWS project in Q3.

Tip: never list a tool you cannot answer two follow-up questions on. If a tool is lab-only, say "I have used it in hands-on labs" - honesty builds more trust than a long list.

> ***What to say to the interviewer:*** *Answer by category, give one real example for your strongest tools, and clearly separate production experience from lab experience. That honesty makes your strong areas more believable.*

Q5. Which AWS services are you proficient in, and how many have you worked with?

In simple words: Nobody knows all 200+ AWS services. They want to hear the core ones you actually used, and what you did with them.

Detailed answer

Pick about 8-12 and attach a verb to each:

- **EKS / EC2** - run and troubleshoot workloads and monitoring servers.
- **Lambda + EventBridge** - scheduled daily health and SLA reports, alert relays, external health checks.
- **CloudWatch + SNS** - alarms for RDS, ElastiCache, MSK; routing alerts.
- **RDS, ElastiCache, MSK** - monitor health, disk, connections, lag.
- **IAM** - roles for Lambda and EC2 instead of access keys (for example replacing an IAM user with an instance role).
- **S3** - storage, lifecycle, cost cleanup.
- **VPC / Security Groups** - understanding connectivity when checks fail.

Example: Sample answer - adapt to your own words:

```
I have worked with around 12 AWS services. Day to day I use EKS, EC2, CloudWatch,
Lambda and EventBridge - for example our daily infrastructure reports are Lambda
functions on an EventBridge schedule that check nodes, RDS, Redis and MSK and post
to Teams. I also monitor RDS, ElastiCache and MSK, and I moved one monitoring server
from stored IAM user keys to an instance role for security.
```

What to say to the interviewer: *Name a realistic number, then give real usage for the top four or five. Mentioning a security improvement like replacing access keys with IAM roles shows maturity.*

Q6. What have you achieved in your career so far?

In simple words: They want proof of impact, not a job description. Pick 3-4 achievements, and for each say the problem, what you did, and the result.

Detailed answer

Sample answer - adapt to your own words (short STAR for each):

1. **Zero-cost alerting that did not depend on a blocked tool** - S/T: the planned on-call tool became unavailable, so alert delivery was at risk. A: built Teams-direct alerting with small Lambda relays for CloudWatch alarms, and pointed Alertmanager's native Teams integration at the channels, with a backup of the old routes. R: alerts kept flowing across projects at no extra cost.
2. **Found and fixed a silent alert-delivery failure** - S/T: new category-based alert channels looked configured but alerts were not arriving. A: tested each channel with a real test message and found empty message templates (and some disabled channels). R: delivery restored and verified on all four environments. The same habit later caught a broken template on the API error alert contact points.

3. **Observability and cost audit** - S/T: logging, metrics and tracing were reviewed across five environments. A: documented each stack and found orphaned volumes, PVCs and frozen buckets. R: identified recurring monthly waste (on the order of tens of dollars per environment) for the owners to clean up, plus real gaps such as broken tracing.
4. **SLA dashboards and reporting across 4 projects** - A: built live Grafana SLA dashboards from the Zabbix SLA API and a daily and monthly SLA reporting routine that explains every minute of downtime with a specific root cause. R: management and clients get consistent, evidence-based availability numbers.
5. **Fixed the daily report automation** - S/T: the scheduled daily report Lambdas were posting the same report three times. A: traced it to automatic Lambda retries on long runs and fixed the retry setting, timeout and message pacing. R: one clean report per day.

What to say to the interviewer: *Pick three or four achievements and give each one a problem, an action and a result. Reliability, cost savings and automation results are what interviewers remember.*

Q7. How much experience do you have handling databases?

In simple words: They want to know if you can keep a database healthy in production - monitoring, backups, failover, connections - not whether you are a DBA who tunes every query. Be honest about which one you are.

Detailed answer

Honest framing (adapt to your real experience): "I have operational and monitoring experience with databases, not DBA-level experience."

- **Engines I work around** - Amazon RDS PostgreSQL; self-managed PostgreSQL on EC2 in one environment; ElastiCache Redis; Kafka on Amazon MSK (and a self-managed Kafka).
- **Monitoring** - CPU, memory, free storage, connections, replica lag, disk alarms in CloudWatch and Zabbix; DB health probes that check the database is actually reachable and answering queries, not just that the host is up.
- **Incidents** - disk-usage alerts and threshold tuning (for example the MSK disk incident), and investigating DB-related downtime for SLA reports.
- **Backup and recovery concepts** - automated RDS backups and snapshots, point-in-time restore, Multi-AZ failover, read replicas for read scaling and how a replica can be promoted.
- **Basics I can do** - connect with `psql`, check active connections and long-running queries, check sizes.

Example:

```
psql -h $DB_HOST -U app -d appdb -c "select count(*) from pg_stat_activity;"
psql -h $DB_HOST -U app -d appdb -c "select pid, now()-query_start as age, state, query
  from pg_stat_activity where state <> 'idle' order by age desc limit 5;"
```

What to say to the interviewer: *I have operational experience with databases from the DevOps side - RDS and self-managed PostgreSQL, Redis and Kafka - covering monitoring, alarms, health probes, backups and failover concepts, but I am not a DBA. I can check connections and slow queries and keep the database healthy and recoverable, and I work with the developers on schema and query tuning.*

Q8. How do you use Linux in your DevOps work, and which distributions have you used?

Also asked as: How much Linux do you know? Which version of Linux do you use?

> **In simple words:** Linux is the ground every DevOps tool stands on. They want to hear that you are comfortable fixing a real server from the command line.

Detailed answer

- **Distributions** - mainly Ubuntu and Amazon Linux (RHEL family) on EC2; container base images like Alpine and Debian slim.
- **Daily use** - SSH into servers, check services with `systemctl`, read logs with `journalctl` and `tail`, check disk/memory/CPU (`df -h`, `du`, `free -m`, `top`), manage cron jobs, and edit configs.
- **Real troubleshooting** - disk filling up on a monitoring server, memory pressure from PHP-FPM worker counts, a hung OS causing a 504 on the web UI.

Example:

```
df -h                                  # which filesystem is full?
sudo du -xh / --max-depth=2 | sort -h | tail
sudo journalctl --disk-usage
sudo journalctl --vacuum-size=500M     # safely shrink old system journals
systemctl status mysqld zabbix-server
```

Sample answer: "When our Zabbix server disk hit about 91%, I found old systemd journals taking large space and reduced usage to about 82% with `journalctl --vacuum-size`, then flagged a large stale data folder for a decision instead of deleting it blindly."

> ***What to say to the interviewer:*** *Name the distributions you use, the everyday commands, and one real Linux incident you solved. Emphasize that you check before deleting anything on a production server.*

Q9. Can you describe the CI/CD workflow in your project?

Also asked as: How do you handle the continuous delivery (CD) aspect in your projects?

> **In simple words:** They want the journey of a code change from a developer's laptop to production, and your part in it. If your role is not the pipeline owner, say so honestly and describe how you support and monitor it.

Detailed answer

Describe the flow clearly (a typical EKS flow - confirm it matches your project before saying it):

1. Developer pushes to Git -> pull request with review.
2. CI (e.g. Jenkins / GitHub Actions) builds, runs tests and code scans, builds the Docker image, tags it with the commit SHA, pushes to Amazon ECR.
3. CD updates the Helm values or manifest with the new tag; deploys to staging first, then to production after approval (via Helm or GitOps with Argo CD).
4. Kubernetes does a rolling update; readiness probes gate traffic.

5. Monitoring after deploy - error rate, latency, pod restarts; rollback with `helm rollback` or `kubectl rollout undo` if needed.

Example: Honest framing - adapt to your own words:

```
In my project the platform team owns the build pipeline, and my part is the
production side: after a deployment, our monitoring shows whether services and SLAs
are healthy, and alerts reach Teams if something breaks. I have also built pipelines
myself in hands-on labs - Jenkins building a Docker image, pushing to ECR and
deploying to Kubernetes with Helm - so I understand each stage end to end.
```

> ***What to say to the interviewer:*** *Walk through code -> build -> test -> image -> registry -> staging -> approval -> production -> monitoring -> rollback, and be precise about which parts you personally own. Honest scoping plus lab experience is far safer than claiming a pipeline you cannot explain in depth.*

Scenario-Based

Q10. Tell me about a recent outage or production incident you handled. What did you learn from it?

Also asked as: Share a real outage incident - what did you learn? Describe an incident and how you responded. Troubleshooting real incidents during on-call. How did you handle outages, communication and postmortems?

> **In simple words:** They are testing how you behave under pressure: do you stay calm, find the real cause with evidence, fix it, and make sure it never happens again?

Detailed answer

Sample answer - adapt to your own words (STAR):

- **Situation** - We had built new category-based alert channels (media types) for Teams across all four environments as part of an alert segregation project. Alerts looked configured, but the team was not receiving some expected alerts.
- **Task** - Find out why alerts were not arriving and restore delivery everywhere, fast, because no alerts means we are blind in production.
- **Action** - Instead of assuming the network or Teams was at fault, I tested each media type with the built-in test button and checked the action logs. I found that the new media types had empty message templates, which silently blocked delivery from the day they were created. In one environment some media types were also disabled. I added the templates, enabled them, and verified delivery with live tests in every environment.
- **Result** - Alert delivery was restored on all four environments and verified end to end.
- **Learning** - "Configured" is not the same as "working". Now every new alert path is tested with a real message before it is marked done, and I think about a heartbeat check for the alerting path itself.

Communication strategy during an outage (they often ask this separately):

- **One incident commander** runs the call and decides; others investigate. This avoids five people changing things at once.
- **Status updates at a fixed cadence** (for example every 15-30 minutes) in one channel and on the status page, even when the update is "no change, still investigating".

- **Stakeholder-friendly language** - impact, what we are doing and the next update time, not internal jargon. Technical detail stays in the engineering channel.
- **Postmortem within 48 hours** - blameless, with a timeline, root cause and owned action items, shared with everyone affected.

What to say to the interviewer: *Tell one incident in STAR form with a clear evidence-based root cause, your specific actions, a verified result and a lasting process change. The learning is what separates a senior answer from a junior one.*

Q11. Tell me about a mistake you made in a DevOps project and what you learned from it.

In simple words: Everyone makes mistakes. They want to see that you own it, fix it quickly and change your process - not that you are perfect.

Detailed answer

Sample answer - adapt to your own words (STAR):

- **Situation** - We moved alerting away from an older on-call tool setup, and I reported that the old setup was no longer active.
- **Task** - Make sure there was exactly one reliable alert path to Microsoft Teams.
- **Action** - Later I noticed duplicate alerts in Teams. When I investigated, I found the old setup was still live, with webhooks still posting alongside the new Lambda relay. My mistake was assuming it was decommissioned without checking every remaining integration. I disabled the old webhooks, fixed a forwarding bug, confirmed only one path remained, and corrected my earlier report to the team.
- **Result** - Duplicate alerts stopped, and the team had an accurate picture again.
- **Learning** - I now verify decommissioning with evidence (list every integration or webhook and confirm zero traffic) before calling anything "removed", and I correct the record openly as soon as I find an error.

What to say to the interviewer: *Pick a real but recoverable mistake, own it plainly, show how you fixed it and corrected the record, and end with a concrete process change. Avoid fake mistakes like "I work too hard".*

Q12. What is the hardest technical issue you have faced, and how did you solve it?

In simple words: They want to see your problem-solving method: how you narrow down an unclear problem step by step until you prove the cause.

Detailed answer

Sample answer - adapt to your own words (STAR):

- **Situation** - In one of our staging environments we had deployed Grafana Beyla (eBPF auto-instrumentation) for distributed tracing, but the tracing backend was receiving no spans. Nothing crashed, it just silently produced no data.
- **Task** - Find why tracing was not working so we could use it as a model for production.

- **Action** - I checked each hop in order: were the Beyla pods running, were they discovering services, could they reach the collector endpoint. The logs pointed to the exporter endpoint. The cluster is IPv6, and the collector address was not written in the bracketed IPv6 format that URLs require, so the export silently failed. I fixed the endpoint format and redeployed.
- **Result** - Spans started flowing, and I confirmed thousands of spans (about 8,000) arriving in the backend.
- **Learning** - For silent failures, walk the data path hop by hop and check each boundary; and IPv6 clusters have their own gotchas, like brackets in URLs.

What to say to the interviewer: *Choose a problem where the cause was not obvious, then show the hop-by-hop method you used to isolate it and the proof that your fix worked. The method matters more than the tool.*

Q13. Tell me about a time you had to deliver something quickly without all the time or resources you needed.

Also asked as: How do you handle tight deadlines?

In simple words: They want to see prioritization: you found the smallest solution that solved the real risk, delivered it, and did not cut corners on safety.

Detailed answer

Sample answer - adapt to your own words (STAR):

- **Situation** - Our planned migration to a new on-call service was delayed, which meant our phone/paging path was not dependable. We still needed production alerts reaching the team immediately.
- **Task** - Keep critical alerts flowing without new paid tools or waiting for the migration.
- **Action** - I first checked what we already had, and found we could send alerts straight to Microsoft Teams. I built small Lambda relay functions for CloudWatch alarms and pointed Alertmanager's Teams configuration directly at the channels. I kept the Lambdas on minimal permissions, took a backup of the old routes so we could restore later, and tested every path.
- **Result** - Built and tested quickly, at no extra cost, with no dependency on the delayed tool.
- **Learning** - Under time pressure, reuse what already exists, deliver the simplest thing that covers the risk, and keep a rollback path.

What to say to the interviewer: *Show that you prioritized the real risk, reused existing tools, delivered a simple tested solution with a rollback plan, and communicated trade-offs. Speed without testing is not the story they want.*

Q14. What would you do if you had to work on a tool or technology you have never used? Share a time you picked up a new tool quickly.

Also asked as: How do you learn a new tool quickly?

In simple words: Technology changes every year. They want to see a learning method: official docs, a small lab, then a real but low-risk task, and asking for help early.

Detailed answer

My approach:

1. Understand the problem the tool solves and the basic concepts (official docs, quickstart).
2. Build a tiny working lab in a non-production place.
3. Apply it to a real small task, and compare my work with best practices.
4. Ask experienced teammates early; document what I learned for the team.

Sample answer - adapt to your own words (STAR):

- **Situation** - We needed monthly SLA dashboards in Grafana with data from the Zabbix SLA API, which our Grafana did not show natively.
- **Task** - Learn a way to pull that API data into Grafana.
- **Action** - I learned the Grafana Infinity data source (querying JSON APIs), tested queries on one small panel, understood its parser options and field mapping gotchas, then built the dashboards through the Grafana API and saved the build scripts so they can be regenerated.
- **Result** - Live SLA dashboards for the environments, repeatable instead of hand-made.

What to say to the interviewer: *Describe your learning loop - concepts, small lab, real low-risk task, ask and document - and back it with one real example where you became productive quickly. That shows you can handle any tool they use.*

Q15. How would you convince your team or manager to adopt a new tool or process?

Also asked as: How do you convince stakeholders?

In simple words: People adopt change when they see their own problem solved with low risk. Bring data, a small proof, and a way back if it fails.

Detailed answer

Steps:

1. **Start with the problem, not the tool** - with numbers: time wasted, incidents missed, money spent.
2. **Show options** - including "do nothing", with cost and effort for each.
3. **Small proof of concept** - on one environment or one team, with success criteria agreed in advance.
4. **Address risks** - cost, learning curve, lock-in, rollback plan.
5. **Share results and roll out gradually**, with documentation.

Sample answer - adapt to your own words (STAR):

- **Situation** - Alert delivery depended on a paid external on-call tool that had become unreliable for us.
- **Action** - I compared options and proposed a Teams-direct design at no extra cost using Lambda relays and Alertmanager's native Teams integration, built it, and showed test alerts arriving in the real channels, with a backup of the old configuration for rollback.
- **Result** - The team accepted it because it solved the immediate problem, cost nothing and was reversible. Similarly, my observability audit presented monthly cost waste with exact items, which made the cleanup decision easy for the owners.

What to say to the interviewer: *I lead with the problem and data, offer options with costs, prove the idea on a small scale with agreed success criteria, and include a rollback plan. People say yes to low-risk, proven improvements.*

Q16. How do you handle conflict or disagreement within your team?

Also asked as: Googliness / culture fit questions.

In simple words: They want to know you disagree respectfully, focus on facts and the shared goal, and can commit to a decision even if it was not your idea.

Detailed answer

Approach:

- Talk directly and privately first, and listen to understand their reasons.
- Move from opinions to evidence: logs, metrics, a quick test.
- Focus on the shared goal (production stability, the customer, the deadline).
- If still stuck, agree on criteria and escalate together to the lead, not around each other.
- Once decided, "disagree and commit" and support the decision fully.

About "Googliness" and culture-fit rounds: they test humility (admitting mistakes, giving credit), collaboration (helping others succeed), doing the right thing when nobody is watching (for example refusing to use pasted credentials or skip a check), and comfort with ambiguity (making progress when requirements are unclear). Answer with real examples, not adjectives.

Example: Sample answer - adapt to your own words:

```
When several SLA breaches happened on the same day, the first explanation given was
a single common cause for all of them. I felt some did not fit, so instead of
arguing, I checked each breach against the activity logs and timelines. It turned
out most matched the common cause, but one was a separate new issue. Presenting
evidence per breach made it a fact discussion rather than a disagreement, and the
report ended up more accurate.
```

What to say to the interviewer: *I keep disagreements private, respectful and evidence-based, anchored on the shared goal, and escalate together only if needed. Once a decision is made I commit to it fully.*

Q17. How do you work across teams (developers, QA, platform, business)?

In simple words: DevOps is a bridge. They want to hear that you communicate clearly, speak each team's language, and make shared information easy to find.

Detailed answer

- **Shared visibility** - dashboards and reports that every team can read (for example SLA dashboards per project).
- **Clear tickets** - Jira stories with context, evidence and expected outcome, assigned to the right owner.
- **Speak their language** - developers want logs, traces and exact errors; managers want impact, downtime minutes and risk; clients want availability and root causes.

- **Proactive communication** - during incidents, regular short updates; after, a shared RCA.
- **Make it easy** - small improvements for other teams, such as adding a pod filter dropdown to the shared Loki logs dashboard so developers can find their pod's logs quickly.

> ***What to say to the interviewer:*** *I make information shared and self-service through dashboards and clear tickets, tailor the message to each audience, and communicate early during incidents. Small improvements that save other teams time build trust quickly.*

Q18. Give an example of taking ownership beyond your assigned task.

> **In simple words:** Ownership means you do not stop at "my part is done". If you see a risk, you raise it or fix it, and you follow through until it is closed.

Detailed answer

Sample answer - adapt to your own words (STAR):

- **Situation** - A ticket said a database health probe had been deployed to all four environments.
- **Task** - My task was only a related monitoring item, but I noticed alerts that should exist for two environments were never firing.
- **Action** - I checked the servers themselves and found the probe was deployed only on two environments; the other two had nothing, including the environment whose outage had created the ticket in the first place. I documented the evidence and raised it so the gap could be closed.
- **Result** - The team got an accurate picture of coverage instead of a false sense of safety.

Other small ownership habits: verifying that alerts really arrive, flagging cost waste found during audits, and correcting records when something turns out different than reported.

> ***What to say to the interviewer:*** *Share a time you went beyond the ticket to verify reality, found a real gap, and followed through until it was fixed or clearly handed over. Ownership is about outcomes, not task boundaries.*

Q19. Can you design, secure, scale, troubleshoot and own production systems?

> **In simple words:** This is a confidence question. Answer with a short, structured "yes, here is how" across each word, backed by one real example each - and be honest about what you are still growing.

Detailed answer

Structure by the five words:

- **Design** - I think about failure first: redundancy, alert paths that do not depend on one tool, and simple, low-cost designs (for example a Teams-direct alerting architecture).
- **Secure** - least privilege IAM, roles instead of stored keys, not pasting credentials in chat or code, secrets in a manager, and flagging open security groups.
- **Scale** - automation over manual work (scheduled reports and checks across four environments), and understanding autoscaling with HPA/KEDA and Karpenter.
- **Troubleshoot** - hop-by-hop, evidence-based root cause (the empty alert template issue, the IPv6 tracing endpoint bug, a hung OS causing a 504).

- **Own** - on-call style ownership of monitoring for four production environments, SLA reports explaining every minute of downtime, and follow-up until issues are closed.

Be honest about the gap: "I am strongest in observability and incident response; I am actively building depth in Terraform and CI/CD with hands-on labs."

> ***What to say to the interviewer:*** *Yes - and I would answer each word with a real example: failure-first design, least-privilege security, automation for scale, evidence-based troubleshooting and end-to-end ownership of monitoring for four production environments. I would also say clearly which area I am still deepening, because that honesty builds trust.*